

ADVANCED FUNCTIONS

Chapter 8

Copyright © 2017 STEM Course Prep



MORE FUNCTION CONCEPTS

- Anonymous functions: simple one-line functions
- Function handles: used to refer to functions
- Function functions: functions to which other functions are passed (in the form of function handles)
- Variable number of input and/or output arguments to functions

ANONYMOUS FUNCTIONS

- Scenario: You've got a function $f(x, z) = x^{2.5} + e^z - \sin x$ and you want to calculate $f(x, z)$ at various x and z values
 - You want to be able to call this function like a standard user-defined function where you supply inputs and get an output
 - Since this is a very simple equation we can create an anonymous function
 - Anonymous functions are simple user-defined functions that are defined within your code
 - You do not create a separate function file
 - Can be constructed in the command window, script files, and regular user-defined functions
 - How do we do it?

ANONYMOUS FUNCTIONS

- General form:

name = @ (input arguments) expression

name of the function

@ symbol – indicates anonymous function

list of input arguments separated by commas

- can have zero or multiple arguments

mathematical expression

- should be short and simple
- can include any built-in or other user-defined functions

ANONYMOUS FUNCTIONS

- Command creates the function
- Assigns a **handle** for the function to the name on the left hand side of =
- In other words.... @ indicates a function and you will call this function by using the variable name on the left side.

CALLING ANONYMOUS FUNCTIONS

- Calling an anonymous function:
`name(input arguments)`



Variable name on
left hand side of =



List of input arguments-
Separated by commas

- Benefits of these functions?
 - Reduces the number of m files you have to deal with!
 - Puts all the code in one file

EXAMPLE

- Define an anonymous function for:

$$f(x) = \frac{e^x}{x^2 - 8}$$

```
>> f=@(x) exp(x)./(x.^2-8);
```

Indicates the input

➤ Does not have to have same name in call

```
>> y=1:3;
```

```
>> f(y)
```

Call statement

```
ans =
```

-0.3883

-1.8473

20.0855

f(1)

f(2)

f(3)

FUNCTION FUNCTIONS

- Scenario: Function A uses another function (function B)
 - When function A is executed, it must be supplied with function B
- A function that accepts another function is called a function function
- How do we pass a function into a function function?
 - Function handles

FUNCTION HANDLES

- A function handle provides a way to call the function associated with it
- Can be used with ANY kind of MATLAB function
 - Built-in
 - User-defined
 - Anonymous
- Created by:
 - Typing @ in front of the function name
 - Method for built-in and user defined functions – anonymous functions don't need this as their name is already a function handle
 - Example: @sqrt is the function handle for the built-in sqrt
 - Can also assign a variable name if wanted
 - Example: sqrtHandle=@sqrt
 - Now sqrtHandle can be used to pass the handle

FUNCTION FUNCTIONS

How to write a function function to accept a function handle:

- Function header must include a dummy function name to represent the imported function
 - This dummy function name will be used throughout when the imported function is needed
 - Number of input arguments and output arguments used with the dummy function must match that of the original function handle
-
- Confused? Let's see an example...

EXAMPLE

- Create a function `plot_this` that:
 - makes a plot of an imported function from $a \leq t \leq b$
 - calculates the value of the imported function at $x = a$ and $x = b$

```
function [ out ] = plot_this( F, a, b )  
t=linspace(a,b,100);  
y=F(t);  
out(1)=F(a);  
out(2)=F(b);  
plot(t,y)  
xlabel('t')  
ylabel('f(t)')  
end
```

Notice: 1 output, 1 input
→ function F is being called

Function dummy
variable

EXAMPLE

- Now create the user-defined function and call statement
- Let $f(x) = 10 \sin(x) \cos(x^2)$
- User-defined function Fexample

```
function [ y ] = Fexample( x )  
%user-defined function of f(x)  
y=10*sin(x).*cos(x.^2);  
end
```

Notice: 1 output, 1 input

EXAMPLE

- Call statement in script:

```
values=plot_this(@Fexample, 0, 10)
```

Function handle passes in $f(x)$ equation

Output:

```
>>
```

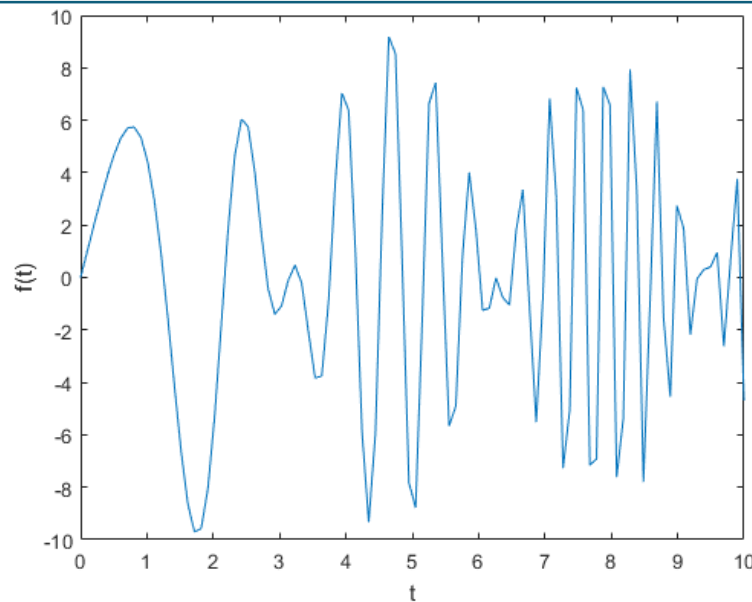
```
values =
```

0

-4.6912

$f(a)$

$f(b)$



FPLOT

- Easy way to plot a function of the form $y = f(x)$:

`fplot(function, limits, 'options')`

Function to be
plotted – use
function handle



2-element vector:
[initial x, final x]



Plot line options:
color, markers, etc.



FPLOT

Example:

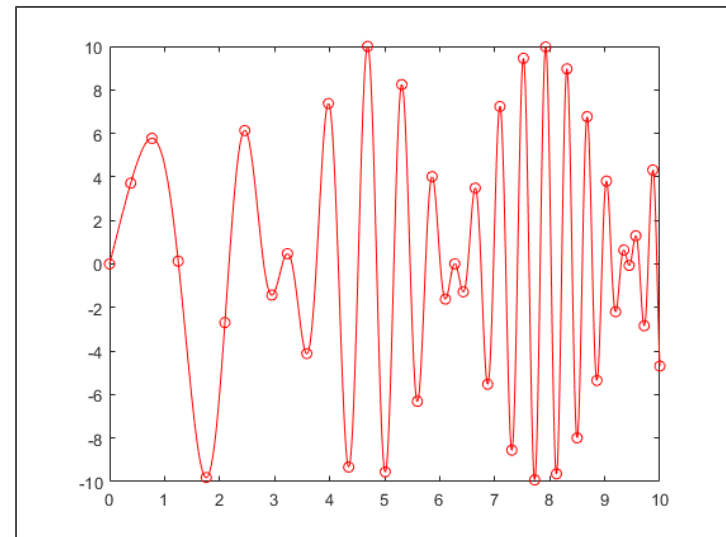
Use $f(x) = 10 \sin(x) \cos(x^2)$ from `Fexample.m`

```
fplot(@Fexample, [0,10], 'ro-')
```

Or

```
hi=@Fexample
```

```
fplot(hi, [0,10], 'ro-')
```



.MAT FILE

- .mat file will save the current workspace so that it can be loaded at a later time
- Easy to save groups of anonymous functions in a .mat file so they can be used later
- Create a .mat file by using the command:
 - `save filename`
- To see what is in the .mat file just type:
 - `load filename`
- Note: this will save the entire workspace. Clear everything out of the workspace first

VARIABLE # OF ARGUMENTS

- What if we need a function that can accept a different number of inputs/outputs depending on the code
- Possible to set this up using varargin and/or varargout
- varargin - cell array* used to store a variable number of input arguments
- varargout - cell array* used to store a variable number of output arguments

* Cell arrays store different kinds of values in the different elements. You could have strings and numbers in the same array for instance. Use { } instead of [] for cell arrays

CONSIDER THIS ¹⁸

- Consider a case where we need a function that will calculate a student's final average.
- Possibilities:
 - The student is required to do 3 exams – each exam worth 33.3%
 - 3 input arguments
 - The student has the option to do a final exam in addition to the required 3 exams– each exam worth 25%
 - 4 input arguments
- Since we don't know ahead of time if the student will take the final we need the code to work for both situations
- This is a case where we need a variable number of inputs

VARIABLE INPUT

- Use **varargin** to store a variable number of input arguments
 - Some or all of the inputs can be stored in varargin
 - I usually use varargin to store the variable inputs only – it results in less typing
- **nargin** – a function that returns the number of input arguments passed to the function
 - This has nothing to do with the length of varargin
 - It counts the number of input arguments in the call statement
- **varargin** is a cell array
 - refer to the elements of it using { }

VARIABLE INPUT

- Choices for function header:
 1. Store some of the input arguments in `varargin`:
`function avg = grade(exam1, exam2, exam3, varargin)`
 - Exams 1-3 will always be provided so they don't have to go in `varargin`
 - `varargin` has 1 element: the final exam - if it's available
 - Since the final is optional it MUST go in `varargin`
 - Refer to the final exam grade as `varargin{1}`
 2. Store all input arguments in **`varargin`**:
`function avg = grade(varargin)`
 - 4 four elements in `varargin`
`varargin{1}=exam1`
`varargin{2}=exam2`
`varargin{3}=exam3`
`varargin{4}=final exam`

FUNCTION HEADER EXAMPLE 1

Some of the input arguments in varargin:

```
function [ avg ] = grade( exam1,exam2,exam3,varargin )
%find a final average
if nargin==3 % nargin=3 because only 3 inputs given
    avg=exam1*.333+exam2*.333+exam3*.333
else % nargin=4 because final exam given
    avg=exam1*.25+exam2*.25+exam3*.25+varargin{1}*.25
end
end
```

FUNCTION HEADER EXAMPLE 2

Store all input arguments in **varargin**:

```
function [ avg ] = grade(varargin)
%find a final average
if nargin==3 % nargin=3 because only 3 inputs given
    avg=varargin{1}*.333+varargin{2}*.333+varargin{3}*.333
else % nargin=4 because final exam given
    avg=varargin{1}*.25+varargin{2}*.25+varargin{3}*.25+varargin{4}*.25
end

end
```

EXAMPLE

Call statements to use with either grade function:

- Without final exam grade:

```
avg=grade(100, 80, 75)
```

```
>>avg =
```

```
84.9150
```

- With final exam grade:

```
avg=grade(100, 80, 75, 60)
```

```
>>avg =
```

```
78.7500
```



Final exam

CONSIDER THIS

- We need to create a function that will allow an instructor to enter an array of exam grades. The function's output will be the mean and, if requested, the standard deviation of the grades
- This gives 2 possible options for output arguments:
 1. One output argument: mean
 2. Two output arguments: mean and standard deviation
- Will need to create a variable output function to accommodate both cases
 - **varargout**

VARIABLE OUTPUT ²⁵

- **varargout** – a cell array used to store a variable number of output arguments
- **varargout** can be used to store some or all of the output arguments
- **nargout** – built-in function that counts the number of output arguments in the call statement
 - Has nothing to do with the size of **varargout**

VARIABLE OUTPUT

- Function header options:
 1. Some of the output arguments in varargout
 - Mean will always be output so that doesn't have to go in varargout
 - Standard deviation is optional so it **must** go in varargout

```
function [avg, varargout] = calc_avg(grades)
```

1 element in varargout:
varargout{1}=std_dev
 2. All output arguments in varargout

```
function varargout = calc_avg(grades)
```

2 elements in varargout:
varargout{1}=avg
varargout{2}=std_dev
- Notice the curly braces – always use these with varargin and varargout

FUNCTION HEADER 1

Some of the output arguments in varargout

```
function [avg, varargout] = calc_avg(grades)
% find mean and std deviation (if needed)
if nargout==1 %only avg in call statement
    avg=mean(grades);
else %nargout=2 in this case - avg and std in call
    avg=mean(grades);
    varargout{1}=std(grades);
end
end
```

FUNCTION HEADER 2

All output arguments in varargout

```
function [varargout] = calc_avg(grades)
% find mean and std deviation (if needed)
if nargout==1 %only avg in call statement
    varargout{1}=mean(grades);
else %nargout=2 in this case - avg and std in call
    varargout{1}=mean(grades);
    varargout{2}=std(grades);
end
end
```

EXAMPLE

- Call statements to use with either `calc_avg` function:

- Average as only output:

```
grades=[45 79 83 98 74 100];  
[avg] = calc_avg(grades)  
>>
```

```
avg =  
    79.8333
```

- Average and standard deviation as outputs:

```
grades=[45 79 83 98 74 100];  
[avg,std_dev] = calc_avg(grades)  
>>
```

```
avg =  
    79.8333  
std_dev =  
    19.9741
```

NOTE

- Optional inputs/outputs should always be the last elements in the `varargin/varargout` cell arrays
- For example, you wouldn't want:
 - `varargout{1}=standard deviation`
 - `varargout{2}=mean`